

CastraNova POS — Product Requirements Document

Version: 2.6 **Date:** 2026-06-02 **Author:** K W G **Status:** Locked — closes all open risks from v2.5 (R1–R6 + §10 timeline ambiguity). Decision Log: D1 → D38. **Implementation contract** for KWG ↔ CastraNova engagement.

1. Executive Summary

CastraNova POS is an inventory tracking and channel-margin reporting system for a B2B refrigeration trading business operating between **Bangkok HQ** (administration) and **Yangon** (warehouse + on-site service). Every machine and high-value spare part is tracked individually (one barcode per piece); commodity parts are tracked by SKU quantity with **FIFO cost layering** — each receipt creates a new purchase batch with its own cost, and consumption draws from the oldest batch first. Every consumption movement records its customer, channel (Sale / Maintenance / Project), and price — feeding a monthly per-channel revenue + margin report. The system works offline at the Yangon warehouse with idempotent sync on reconnect; **all inventory-out recording happens at the warehouse**. Bangkok HQ maintains the catalog, prices, customers, projects, user accounts, and **initiates all Project-channel consumption via Project Pulls** that Yangon staff fulfill; Yangon executes physical receiving, sales, and maintenance.

2. Problem Statement

Current state

- Bangkok HQ has no reliable view of Yangon stock or recent consumption.
- Inventory movements live in spreadsheets and memory; month-end reconciliation is slow and error-prone.
- The Yangon team loses time on paper-based receiving and handwritten service notes.
- Yangon's internet is unreliable; current digital tools fail when the connection drops.
- Channel mix (Sale / Maintenance / Project) and inventory holding period are invisible.

Pain points

1. **No live stock visibility** at Yangon.
2. **No channel separation** in consumption tracking — sales, on-site maintenance, and projects blend together, hiding true margins.
3. **No defensible audit trail** for inventory movements or price changes.
4. **Yangon internet is unreliable** — current digital tools fail when the connection drops.
5. **Holding period for stock is unknown** — slow-movers are not identifiable.

Business impact

- Month-end reporting takes days and is error-prone.
 - Slow-moving stock ties up capital invisibly.
 - Margin decisions are made on incomplete data.
-

3. Goals & Success Metrics

Goal	What we measure	Target	How we check it
See live stock at Yangon	How long it takes to open the warehouse stock	Under 10 seconds, almost every time	Time it with a stopwatch during the

	screen and see what's in stock		pilot
Get monthly margin reports on time	The monthly Sale / Maintenance / Project margin report is ready by the end of the first business day of the following month	By end of Day 1 of the next month	Date stamp on the report
Every inventory movement is traceable	Percentage of inventory movements (receipts, sales, repairs, project pulls, adjustments) that record who did it, when, and why	100%	Pick 50 movements at random each month and check each one has a name, time, and reason
Keep working when the warehouse internet is down	Sales, scans, and movements captured during an 8-hour internet outage at the warehouse	All of them — no lost work	Unplug the warehouse router for 8 hours during the pilot and confirm everything synced afterwards
Spot slow-moving stock easily	How many clicks it takes to see the top slow-movers	One click from the main dashboard	Show the top-20 slow-movers list during the demo
Reduce stock-out incidents at Yangon	Number of times a part runs out of stock per month at Yangon	Month 1 = baseline (no target). Month 2 onward = reduction target set together with CastraNova based on the baseline.	Counted automatically from the system
Smooth sync after offline work	Percentage of offline sync events that finish on their own without needing the admin to step in	95 out of every 100, or better	Counted from the admin's review queue at the end of each month

4. User Personas

Bangkok Admin (HQ)

- **Role:** Bangkok HQ staff (~5 people).
- **Goals:** Maintain the product catalog and prices, manage suppliers, customers, and projects, **create Project Pulls** when inventory is deployed for a customer project, approve large pricing overrides, perform admin-only Stock Adjustments, review monthly reports and customer/project history dashboards.
- **Pain points:** No timely visibility into Yangon stock; manual month-end reconciliation; no clean way to record project consumption separately from sales.
- **Technical proficiency:** Medium — comfortable with web apps and spreadsheets.
- **Usage context:** Daily, on desktop/laptop at the Bangkok office, reliable internet.

Yangon Staff (Receiving, Sales, Maintenance, Pull Fulfillment)

- **Role:** Unified Yangon team (~5 people). The same people handle warehouse receiving, customer sales, on-site service jobs (off-system), and fulfillment of admin-created Project Pulls.
 - **Goals:** Receive incoming shipments, label new stock, process customer sales at the warehouse, record parts consumed for maintenance jobs at the warehouse terminal, fulfill Project Pulls created by Bangkok admin.
 - **Pain points:** Slow paper-based receiving; no easy way to check stock; warehouse wifi is patchy.
 - **Technical proficiency:** Low–Medium — phone-first users at the warehouse.
 - **Usage context:** Multiple times per day at the YGN warehouse barcode terminal. On-site customer visits are off-system — mechanics list needed parts on paper/LINE and warehouse staff process them on return.
-

5. Functional Requirements

Priorities: **P0** (must have for launch), **P1** (important for first version), **P2** (nice to have).

Catalog & Master Data

FR-001: Product Catalog

Description: Bangkok admin maintains the master list of all products — refrigeration machines, high-value spare parts (compressors, motors), and commodity spare parts (filters, fittings, refrigerants).

User story: As a Bangkok admin, I want one place to add, edit, and categorize every product so that Yangon staff can scan and sell consistent items.

Acceptance criteria:

- ☐ Each product has a unique SKU, model name, brand, category, and **tracking mode** — **SERIALIZED** (every piece tracked individually) or **QUANTITY** (SKU stock counter with FIFO batches).
- ☐ Each product category carries a **default tracking mode**; Bangkok admin can override per product when creating or editing.
- ☐ Initial catalog is imported from the existing product list the client will provide (CSV/XLSX, KWG runs the seed import once at deployment).
- ☐ Products can be deactivated by Bangkok admin (no hard delete — preserves history).

Priority: P0

FR-002: Per-Product Pricing (FIFO cost basis)

Description: Each product carries a **purchase cost basis**, a **retail price** (Sale channel), and a **repair price** (Maintenance channel). Project consumption uses cost only — revenue is 0. For QUANTITY-tracked products, the effective purchase cost at any moment is the cost of the **oldest non-depleted batch** (FIFO). When a single consumption spans a batch boundary — for example, a sale of 5 units while the oldest batch has only 3 remaining — the cost **splits proportionally across batches**: 3 units at the old batch's cost + 2 units at the new batch's cost. This is accountancy-standard FIFO (option (a)), explicitly chosen over (b) charging the whole sale at the old batch's price or (c) blocking sales that span batches. For SERIALIZED products, purchase cost is captured per piece at receipt.

User story: As a Bangkok admin, I want distinct retail and repair prices for every product, and I want the cost basis of commodity parts to honour the order they entered stock, so that monthly margin reports

reflect the real economics.

Acceptance criteria:

- ☐ Retail price and repair price both editable by Bangkok admin.
- ☐ Every price change is recorded permanently — auditors can answer "*what was the price of part X on a given date?*"
- ☐ For QUANTITY-tracked products, **no single "purchase cost" column on the product** — cost is derived from the oldest non-depleted `part_batch` row. The dashboard displays "current FIFO cost = oldest batch's cost."
- ☐ **Batch succession is automatic.** When the oldest batch reaches `remaining_qty = 0`, it is marked depleted, and the next batch in receipt order automatically becomes the effective purchase cost. No `price_change` record is created for the succession — per-batch costs are immutable; only the *active* batch changes.
- ☐ **Mid-consumption splits produce multiple cost lines.** A single consumption row that spans two or more batches generates one `cost_line` row per batch consumed (referencing each batch's `id` and the quantity drawn). The customer-facing receipt shows the line at its sale price; the internal margin record carries the split for accurate COGS.
- ☐ For SERIALIZED products, purchase cost is captured per piece at receipt.
- ☐ Yangon staff cannot edit prices; they can only override per-transaction via FR-010.

Priority: P0

FR-003: Suppliers, Customers, Projects

Description: Bangkok admin maintains the authoritative lists of suppliers, customers (Dealer or End Customer), and projects (each with code, name, customer, dates, optional budget).

User story: As a Bangkok admin, I want to maintain authoritative lists so that every transaction can be tagged correctly.

Acceptance criteria:

- ☐ Customer record stores name, country, contact, type (Dealer / End Customer).
- ☐ Project record stores code, name, customer link, start/end dates, status, optional budget.
- ☐ Yangon staff can read customers + projects, and may **inline-create a customer record** during a Sale (per FR-007). Yangon staff cannot create or edit suppliers or projects.
- ☐ **v1 tolerates duplicate customer records** (e.g., from offline inline-creation during sync). Admin merge tool deferred to v1.1 (see D35).

Priority: P0

FR-004: User & Role Management

Description: The system supports two roles: **Bangkok admin** and **Yangon staff**.

User story: As a Bangkok admin, I want to add new staff and assign them the correct role.

Acceptance criteria:

- ☐ Bangkok admin can create, edit, and deactivate user accounts.
- ☐ **Bangkok admin role:** catalog, pricing, suppliers, customers, projects, **Project Pull creation**, override approvals, Stock Adjustment, user management, full customer/project dashboards.

- ☐ **Yangon staff role:** receiving, sales, on-site service tickets (recorded at warehouse), **Project Pull fulfillment**, search, customer/project dashboards (transactions only — no financials).
- ☐ **New user onboarding includes a one-time LINE/Viber bot enrollment step** (per D33) — admin runbook documents the procedure; KWG handles the first 10 users at deployment.

Priority: P0

Receiving at Yangon

FR-005: Serialized Inventory Receipt (machines + high-value parts)

Description: When SERIALIZED inventory arrives at Yangon — machines and designated high-value parts such as compressors and motors — staff scans the supplier's serial number on each piece, the system generates a CastraNova barcode per piece, and a printable label is produced.

User story: As a Yangon staff member, I want to record each incoming serialized item quickly so that every individual piece is trackable from the moment it arrives.

Acceptance criteria:

- ☐ Receipt captures the supplier's serial number and the CastraNova-generated barcode on each piece.
- ☐ System produces a printable label (PDF) per piece — sized for the in-warehouse label printer (4x6 in default; configurable in system settings).
- ☐ Each piece's purchase cost and received date are stored individually.
- ☐ If the supplier didn't print a clean serial, staff types it manually.
- ☐ CastraNova barcode generation algorithm: `CN-{8-char-base32}` derived from the unit UUID (Code128 encodable, human-readable for manual lookup).

Priority: P0

FR-006: Quantity-Tracked Parts Receipt (FIFO batches)

Description: When commodity spare parts (QUANTITY-tracked products) arrive at Yangon, staff scans the product's SKU barcode and enters quantity and purchase cost. **A new purchase batch is created** with its own cost; the batch queues behind any existing non-depleted batches of the same SKU for FIFO consumption.

User story: As a Yangon staff member, I want to add incoming commodity parts to stock without labelling every individual piece, with each receipt forming its own batch so the FIFO order is preserved automatically.

Acceptance criteria:

- ☐ Staff scans the product's SKU barcode (one barcode per product, used at the shelf bin and on the receiving form).
- ☐ Staff enters the quantity received and the purchase cost from the supplier invoice.
- ☐ System creates a new `part_batch` row with:
 - `batch_no` auto-generated as `YYYYMMDD-{SKU}-###` (daily sequence per SKU; e.g. `20260527-FLT-001`).
 - **Batch number sequence is race-safe** under concurrent same-SKU same-day receipts via `pg_advisory_xact_lock(hashtext(date_YYYYMMDD || '-' || sku))` within the receipt transaction (per D36).
 - `received_at` timestamp.

- `received_qty` (set on creation; immutable thereafter).
- `remaining_qty` (= `received_qty` at creation; decremented as consumed).
- `purchase_cost` (per unit, immutable thereafter).
- Optional `supplier_batch_ref` free-text field for the supplier's lot number (nullable).
- ☐ SKU's total quantity-on-hand at YGN = sum of `remaining_qty` across all non-depleted batches.
- ☐ Last-received date is captured per SKU (rough holding-period signal).
- ☐ Missing or extra items vs the supplier manifest are flagged for confirmation.

Priority: P0

Consumption — Channels

Design note (v2.5 carry-over): The data model tracks three channels (`SALE` , `MAINTENANCE` , `PROJECT`), each with its own margin rules. The **Yangon staff consumption screen shows a two-tag segmented control — Sale / Maintenance**. `PROJECT` consumption is **never initiated by Yangon staff**; it is created as a Project Pull by Bangkok admin and fulfilled by Yangon staff via a separate "Pending Project Pulls" queue (see FR-009).

FR-007: Sale (Sales channel)

Description: Yangon staff records the sale of one or more machines and/or parts to a customer.

User story: As a Yangon staff member, I want to record a sale by scanning machines and/or selecting parts with quantities, so that stock is deducted and revenue is captured.

Acceptance criteria:

- ☐ **Sale requires a customer to be selected (Dealer or End Customer); no walk-in or anonymous sales.** Staff can quickly create a new customer record inline if needed.
- ☐ Machines and serialized parts: scan each piece's barcode → system verifies in-stock → mark as sold at that piece's selling price.
- ☐ Parts (QUANTITY-tracked): scan SKU barcode and enter quantity → system deducts from YGN stock following **FIFO across batches**. A single sale line may produce multiple internal cost lines when the quantity spans two or more batches (e.g., 5 units = 3 from batch A @ \$100 + 2 from batch B @ \$120). Each cost line references the source batch.
- ☐ System refuses to oversell (no negative stock).
- ☐ Receipt PDF printable for the customer (operational receipt — not a legal tax document).
- ☐ Works offline at the YGN warehouse; syncs when reconnected. Idempotency key on every sale request prevents double-insert on replay.

Priority: P0

FR-008: On-Site Maintenance (Maintenance channel — recorded at warehouse)

Description: When a customer reports a problem, a Yangon staff member visits the customer site to assess and lists the parts needed. The mechanic returns to the warehouse (or relays the list to warehouse staff). **All parts are scanned out at the YGN warehouse barcode terminal** and physically delivered to the on-site mechanic; nothing is recorded from the customer site.

User story: As a Yangon staff member, I want to record what parts a customer's repair consumed at the warehouse terminal, even when warehouse wifi is patchy, so that stock is deducted correctly and the

maintenance history is preserved.

Acceptance criteria:

- ☐ Service ticket links to a customer (mandatory).
- ☐ Service tickets are **opened and closed at the warehouse** — there is no on-site app workflow.
- ☐ Staff can scan/enter parts used during the repair, with quantities, at the warehouse barcode terminal.
- ☐ Parts consumed are priced at the product's **repair price**, not retail (overridable per FR-010).
- ☐ QUANTITY-tracked parts follow FIFO at consumption time (per FR-007 cost-split rule).
- ☐ If a whole machine is swapped at the customer site, the new unit is recorded at the warehouse terminal as a normal Maintenance-channel exit. **The original sale is not linked in v1**; staff capture any unit-swap context in free-text notes on the service ticket.
- ☐ Ticket can be closed; closing deducts all parts from stock in one transaction.
- ☐ Works offline at the YGN warehouse; syncs when reconnected.

Priority: P0

FR-009: Project Consumption (admin-initiated Project Pull workflow)

Description: Project consumption is **initiated exclusively by Bangkok admin**. When inventory is deployed as part of a customer project, BKK admin creates a **Project Pull** listing the project, customer, and line items (specific `SERIALIZED` serials or `QUANTITY` SKU + quantity). The pull enters `PENDING` state and appears in a Yangon staff "Pending Project Pulls" queue. Yangon staff fulfills the pull by scanning each item at the warehouse barcode terminal, which finalizes a `PROJECT`-channel consumption (cost only, revenue = 0).

Acceptance criteria:

- ☐ **Yangon staff consumption screen shows two tags only: Sale, Maintenance.** No Project tag is visible to staff.
- ☐ Bangkok admin creates a Project Pull with:
 - `project_id` (FK; customer is auto-derived from the project).
 - One or more line items: each is `product_id` + (specific `unit_serial` for `SERIALIZED`, or `quantity` for `QUANTITY`).
 - Optional admin notes.
- ☐ Pull state machine: `PENDING` → `FULFILLED` (all lines scanned), `SHORT` (one or more lines partially fulfilled or unavailable), `CANCELLED` (admin-cancelled before fulfillment).
- ☐ On creation, pull state = `PENDING` and appears in YGN staff "Pending Project Pulls" queue.
- ☐ Yangon staff fulfills by scanning each item at the warehouse barcode terminal. Each scan validates stock; `QUANTITY` parts decrement FIFO across batches (per FR-007 cost-split rule).
- ☐ On full fulfillment, pull state → `FULFILLED`. One `PROJECT`-channel consumption record per line is written; cost is recorded per FIFO; **revenue = 0**.
- ☐ If a line is short on physical stock, staff can mark it **partial / short** with the actual quantity fulfilled. Pull state → `SHORT`; BKK admin notified via LINE + Viber (FR-018).
- ☐ BKK admin can cancel a `PENDING` or `SHORT` pull; cancellation is audit-logged and notifies the staff who saw it in queue (if any).
- ☐ Audit record on every consumption names **both users**: admin creator + staff fulfiller (plus timestamps for each).
- ☐ Works offline at the YGN warehouse; syncs when reconnected.

Priority: P0

FR-010: Pricing Overrides with Audit Trail

Description: Yangon staff can override the default price on a Sale or Maintenance line when needed, but every override is recorded with a mandatory reason. Overrides above the threshold (default 5% deviation) require Bangkok admin co-sign.

Acceptance criteria:

- ☐ Override on any Sale or Maintenance line requires a non-empty reason.
- ☐ The deviation threshold that triggers Bangkok co-sign is **set by Bangkok admin** in system settings (default: **5%**, editable any time).
- ☐ Override above the current threshold routes to a Bangkok admin approval queue before the transaction completes.
- ☐ Yangon staff sees "awaiting approval" state on the held transaction; admin approval/rejection posts back and unblocks/blocks the txn.
- ☐ Monthly override exception report lists every override, sorted by deviation size.
- ☐ Project Pulls do not use overrides (revenue is always 0).

Priority: P1

Inventory Management

FR-011: Stock Adjustment (Bangkok admin only)

Description: Bangkok admin can adjust inventory directly to reconcile damaged stock, physical recount discrepancies, supplier returns, or any other gap between system and physical reality. **Yangon staff cannot see or invoke this action.** Every adjustment is audit-logged with actor, timestamp, and mandatory reason.

Acceptance criteria:

- ☐ Bangkok admin selects the item to adjust — a specific serialized piece **or** a SKU + quantity delta — and provides a mandatory reason.
- ☐ Adjustment is one-tap (no two-step approval); audit-logged immediately on submit.
- ☐ Serialized pieces can be moved to a terminal `ADJUSTED_OUT` state (visible in lifecycle history; not in active stock).
- ☐ QUANTITY-tracked SKUs accept positive or negative deltas:
 - ☐ **Negative deltas consume oldest batches first (FIFO).** Affected `part_batch` rows are recorded on the adjustment for audit.
 - ☐ **Positive deltas** create a new `part_batch` row labelled `YYYYMMDD-{SKU}-ADJ-###` with the admin-supplied cost basis.
- ☐ Negative-beyond-current-stock is blocked.
- ☐ Yangon staff role cannot see or invoke this action.
- ☐ All adjustments appear in the monthly audit log under a dedicated "Adjustments" filter.
- ☐ No notifications fire — this is a quiet, admin-side action.

Priority: P1

Reports & Visibility

FR-012: Stock-on-Hand Dashboard

Description: Live view of current inventory at Yangon — serialized items by piece, QUANTITY-tracked parts by SKU.

Acceptance criteria:

- ☐ Serialized items list shows serial/barcode, model, location/status, received date, purchase cost.
- ☐ QUANTITY-tracked parts list shows SKU, name, current quantity-on-hand at YGN (sum of `remaining_qty` across batches), last-received date, **current FIFO cost (oldest non-depleted batch's cost)**.
- ☐ Each QUANTITY row expands to a per-batch breakdown: `batch_no` , `received_at` , `received_qty` , `remaining_qty` , `purchase_cost` , optional `supplier_batch_ref` .
- ☐ Searchable and filterable by category, supplier, customer.

Priority: P0

FR-013: Monthly Channel Margin Report

Description: End-of-month report showing revenue, cost of goods sold (COGS), and margin broken out by channel (Sale, Maintenance, Project).

Acceptance criteria:

- ☐ Report lists revenue, COGS, and margin per channel.
- ☐ COGS for QUANTITY consumption is the sum of batch-cost lines (FIFO).
- ☐ Drill-down by product, customer, project supported.
- ☐ Available on-screen + PDF + Excel export.

Priority: P0

FR-014: Inventory Holding Period Report

Description: Shows how long inventory has been in stock; identifies slow-moving inventory.

Acceptance criteria:

- ☐ Per serialized piece: days from received to today (or to exit date if sold/consumed/adjusted).
- ☐ Per QUANTITY-tracked batch: days from `received_at` to today (per-batch granularity).
- ☐ Per QUANTITY-tracked SKU rollup: days since oldest non-depleted batch was received, plus current quantity on hand.
- ☐ Slow-mover threshold configurable (e.g., > 90 days highlighted).

Priority: P1

FR-015: Search & Lookup

Description: Search by serial/barcode (for serialized pieces) or by SKU (for QUANTITY parts).

Acceptance criteria:

- ☐ Serialized-piece search: full lifecycle (received → sold/consumed → any maintenance → any adjustment), with originating Project Pull if applicable.

- ☐ SKU search: per-batch history + consumption history (all channels, with customer/project, and **which batch(es) each consumption drew from**) + current quantity-on-hand.

Priority: P1

FR-016: Low-Stock Alerts (per SKU)

Description: Each spare-part SKU has its own minimum stock level set by Bangkok admin. When the total quantity-on-hand drops below the threshold, an alert fires.

Acceptance criteria:

- ☐ Bangkok admin can set a minimum-stock-level per SKU (single-row edit + **bulk-edit screen** for setting many at once).
- ☐ When YGN quantity-on-hand drops below the threshold, an alert fires via LINE and Viber.
- ☐ Dashboard always shows the current low-stock SKU list.

Priority: P1

FR-017: Export to PDF and Excel

Description: Every list view and report can be exported to PDF and Excel.

Acceptance criteria:

- ☐ One-click PDF export from any list/report view.
- ☐ One-click Excel export from any list/report view.
- ☐ Export endpoints covered: stock-on-hand, channel margin, holding period, override exception, search results, customer detail, project detail, audit log, low-stock list, adjustments history.

Priority: P1

Notifications

FR-018: LINE and Viber Notifications

Description: Operational alerts pushed to LINE and Viber. Each user can opt in/out per channel and per event type.

Acceptance criteria:

- ☐ Trigger events:
 - Low-stock alert (per FR-016).
 - Large pricing override pending (per FR-010).
 - **Project Pull fulfilled** — notifies BKK admin (the pull's creator).
 - **Project Pull marked short** — notifies BKK admin (the pull's creator).
- ☐ Per-user opt-in/out for each channel (LINE, Viber) and each event type.
- ☐ Failed deliveries retry with exponential backoff (tenacity, 3 attempts: 1s/5s/25s); permanent failures logged to `notification_log` for admin review.
- ☐ **Per-user bot enrollment is a one-time onboarding step** (per D33). LINE Messaging API + Viber Bot API both require the user to first-message the bot to capture their user-id before push works. KWG performs this for the initial 10 user accounts at deployment; subsequent user additions follow the admin runbook.

Priority: P1

Audit & Data Integrity

FR-019: Append-Only Audit Trail

Description: Every inventory movement, price change, and adjustment is recorded permanently. Corrections happen via compensating records — nothing is deleted or quietly edited.

Acceptance criteria:

- ☐ Every movement records WHO did it, WHEN, and (where applicable) WHY.
- ☐ Project Pull consumptions record both the **admin creator** (`created_by_user_id` , `created_at`) and the **staff fulfiller** (`fulfilled_by_user_id` , `fulfilled_at`).
- ☐ Each cost line on a QUANTITY consumption references the source `part_batch` (immutable audit chain from movement → batch → original receipt).
- ☐ Movements cannot be deleted or edited after the fact (DB-level `REVOKE UPDATE, DELETE` on ledger tables).
- ☐ Corrections happen via new compensating entries.
- ☐ Admin can view a filterable audit log by user, date, event type, SKU, or batch.

Priority: P0

Customer & Project Analytics

FR-020: Customer & Project Detail Dashboard

Description: Drill-down view of a single customer (with nested projects) for historical analysis.

Acceptance criteria:

Customer detail page:

- ☐ Header: customer name, type (Dealer / End Customer), country, contact.
- ☐ Lifetime totals (BKK admin only): Sale revenue / COGS / margin; Maintenance revenue / COGS / margin; Project COGS (revenue = 0).
- ☐ Transaction list: paginated, filterable by date range and channel. Visible to both roles.
- ☐ Nested **Active projects** section: each row shows project code, name, budget (admin only), consumed cost-to-date (admin only), % of budget consumed (admin only).
- ☐ Nested **Closed projects** section: same columns, collapsed by default.
- ☐ Action: export to PDF + Excel (admin only; staff sees the export buttons disabled).

Project detail page:

- ☐ Header: project code, name, customer, dates, status, budget (admin only).
- ☐ Cost-to-date and budget-vs-actual chart (admin only).
- ☐ Consumed items list with batch attribution. Cost column visible to admin only.
- ☐ Pull history with state per pull.

Role-tiered access:

- ☐ **Bangkok admin** sees full data. **Yangon staff** sees transactions and item history only; financial fields are absent (not blank).

- ☐ Backend enforces redaction (two `*Public` Pydantic variants); frontend cannot reveal hidden fields via DOM inspection.

Navigation:

- ☐ Accessible from Customer list (both roles). Project list accessible to admin only.

Priority: P1

6. Non-Functional Requirements

6.1 Performance

- Page load (P95): < 2s on warehouse 4G; < 1s on BKK fibre.
- Stock-on-Hand dashboard P95: < 10s (per §3 metric).
- Monthly margin report render P95: < 5s for a single-month window.
- Concurrent users: 10 sustained, 20 peak.

6.2 Security & Auth

- JWT bearer, access token 30 min, refresh token 7 days in `httpOnly secure sameSite=lax` cookie.
- Two roles, enforced server-side via FastAPI dependency.
- Auth endpoints rate-limited 5 attempts / 15 min / IP.
- HTTPS only (Traefik + Let's Encrypt).
- Daily `pg_dump` to Hostinger object storage, 30-day retention, monthly restore drill in staging.
- `SECRET_KEY` rotated annually.

6.3 Offline operation (YGN warehouse) — revised per D32

- System works at the YGN warehouse when wifi is patchy or unavailable. All inventory-out recording happens at the warehouse.
- Scans, sales, maintenance entries, and Project Pull fulfillments are saved locally first via TanStack Query mutation queue persisted to IndexedDB.
- On reconnect, mutations replay in submission order. Idempotency keys (client-generated UUIDs) prevent double-writes.
- **Locally cached data is NOT app-level encrypted (D32).** Rationale: cached data is operational inventory only — no PII beyond customer name/contact which is already in the synced catalog. Device-level encryption (iOS/Android, modern laptop full-disk encryption) plus the 12h auto-logout and 7-day queue cap provide adequate protection without the browser-close data-loss risk of session-keyed encryption.
- 12h auto-logout clears the mutation queue + query cache via `queryClient.clear()` + IDB wipe.
- Queue capped at 7 days; mutations older than 7 days are flagged with `STALE` and routed to admin review on replay.
- Conflict detection: if two clients act on the same `SERIALIZED` unit (e.g., one sells while another fulfills a project pull), server-side first-write-wins (FIFO lock on `unit` row); loser receives 409 with current state; frontend surfaces "already sold by [X] at [time]" and routes to admin conflict-review queue if the user cannot resolve inline.

6.4 Language and currency

- English only in v1.
- Thai Baht (THB) only in v1.
- All monetary fields stored as `NUMERIC(12,2)`.

6.5 Hardware — locked per D34

Reference Bluetooth barcode scanners (KWG-QA'd):

- **Honeywell Voyager 1602g** (BT HID mode, 1D barcodes)
- **Zebra DS2208** (USB/BT HID mode, 1D + 2D barcodes)

Both reference models are paired as HID keyboard wedges; the scanner injects keystrokes into the focused input followed by a configurable terminator (CR by default). Frontend listens for the terminator to commit the scan.

Other HID-mode scanners may work but are supported on **best-effort only**; bring-up of a non-reference model is billable at KWG day rate.

Label printer: Any USB or network printer that accepts 4x6 in PDF. KWG-validated against Brother QL-820NWB (4x6) and Zebra ZD220 (configurable label sizes). Network printer setup is a one-time deployment task.

Phone-camera fallback: YGN staff may scan with the phone camera via the in-browser scanner (`html5-qrcode lib`) when a Bluetooth scanner is unavailable. Slower than HID; not the primary path.

6.6 Hosting and uptime

- Hostinger VPS, Singapore region. Sizing baseline: 4 vCPU / 8 GB RAM / 100 GB SSD ("KVM 4" or equivalent).
- Daily database backups, 30-day retention, monthly restore tests.
- Uptime target: 99% (excluding planned maintenance windows announced ≥ 48h ahead).

6.7 Audit

- All transactions audit-tagged with user, timestamp, and (where applicable) reason.
- Monthly override-exception report and Stock-Adjustment history report available to Bangkok admin.
- Project Pull audit shows both admin creator and staff fulfiller on every consumption.

6.8 Observability

- Sentry for error tracking (frontend + backend); sample rate 100% in v1.
- Backend access + error logs retained 30 days on VPS, rotated daily.
- LINE/Viber notification delivery failures logged to `notification_log` ; admin reviews weekly.

7. UI/UX Requirements

Bangkok admin (desktop-first)

- **Dashboard:** stock on hand (serialized + QUANTITY with per-batch drill-down), alerts, pending approvals, pending/short Project Pulls awaiting admin action.
- **Catalog management:** products, suppliers, customers, projects, users.
- **Project Pulls section:** create new pull, pull history with state filter, cancel pending pulls.
- **Approval queue** (large pricing overrides).
- **Stock Adjustment** (admin-only).
- **Customer detail dashboard** with nested projects (FR-020) — full financial view.
- **Reports section** (monthly margin, holding period, audit log, override exceptions, adjustments history, exports).
- **System settings** (override threshold, low-stock default, label printer config).

Yangon staff (phone-first, warehouse-only)

- **Home screen:** receive, sell, on-site service, pending project pulls, search.
- **Receive flow (serialized):** scan supplier serial → confirm details → print label per piece.
- **Receive flow (quantity):** scan SKU → enter quantity + cost → system creates new `part_batch`.
- **Sale flow:** select customer (required) → two-tag segmented control (Sale / Maintenance), Sale default → scan / select items → confirm.
- **Service-ticket flow (Maintenance):** open ticket at warehouse → scan/enter parts → free-text notes → close at warehouse.
- **Pending Project Pulls screen:** list of admin-created pulls in `PENDING` state; tap → view lines → scan each item → mark fulfilled or short.
- **Customer detail dashboard** — transaction-only view; financial columns hidden.
- **Offline indicator** clearly visible at all times. Queue counter shows pending sync count.

Common behaviors

- Loading and empty states explicitly designed.
- Errors surfaced in plain language (e.g., *"This unit is already sold"* — never an error code). Backend 409 reasons map to translated user-facing strings.
- Phone-camera scan modal accessible as a fallback when no BT scanner is paired.

8. Edge Cases & Error Handling

What could go wrong	What the system does
Two staff sell the same machine — one of them is offline	When the offline one comes back online, the system spots the clash and shows <i>"already sold by [name] at [time]"</i> so they can fix it.
Staff tries to sell more parts than are in stock	The sale is blocked. The screen shows exactly how many are actually available.
Supplier's serial number sticker is messy or unreadable	Staff types it in by hand. The system accepts it like any scanned serial.
A big discount needs Bangkok approval but the admin isn't around	The sale waits in "awaiting approval" status. Once admin approves on LINE / Viber, the sale goes through.
Yangon finds damaged stock	Staff messages Bangkok on LINE or phone; Bangkok admin writes it off using the Stock Adjustment screen with a reason.
A sale empties the oldest batch and dips into the next one	The customer just sees one line on their receipt (e.g. <i>"5 filters @ \$120 each"</i>). Behind the scenes the system uses both batches' costs so the margin report stays accurate.
A new batch arrives while the old batch still has stock	The new batch waits in line. Sales keep using the old batch's price until it's fully sold out, then automatically switch to the new one.
Bangkok creates a Project Pull for a machine that Yangon happens to be selling at the same moment	Whoever saves first wins. The other person sees <i>"already sold"</i> or <i>"already pulled for project X"</i> and can pick a different item.

Yangon staff's phone runs out of battery while offline work is still pending	When the phone is back on, the pending work is still there. It syncs the moment internet returns.
The Bluetooth scanner sends odd extra keystrokes	The system reads only the actual barcode and ignores anything extra.

9. Out of Scope for v1

Carried over from v2.5:

- Customer-facing invoicing or legal tax documents.
- Warranty start/end tracking per unit.
- Labor charges on repairs.
- Tracking inventory while still in Bangkok or in transit.
- Machine returns to the warehouse for service.
- Multi-currency or foreign-exchange tracking.
- Customer self-service portal.
- Native mobile app (PWA only).
- Real-time multi-user live updates (sync-on-reconnect is sufficient).
- Direct integration with accounting or tax systems.
- Multi-language UI.
- Staff-facing scrap workflow.
- Machine-swap → original-sale linkage in the data model.
- Staff-initiated Project consumption.
- On-site consumption recording.
- Returns-to-stock flow.
- Weighted-average cost option.
- Per-batch physical labelling of commodity parts.

Deferred to v1.1 (separate engagement):

- **Admin customer-merge tool** for duplicate-customer reconciliation (D35).
- **Browser-side IDB encryption** if Hostinger/CastraNova security policy later demands it (D32; v1 chose device-level encryption).
- **Stock-out incident dashboard view** as a first-class report (currently derivable from `unit_movement` + `part_movement` queries; FR-016 alerts cover the operational need).
- **Bulk catalog re-import** (v1 supports one-time seed import; ongoing additions are single-row).
- **SLA / 24x7 support tier.** Post-launch support window (30 calendar days at no charge) covers bug fixes only; further support requires a retainer or T&M.

10. Build Phasing — revised per D37

Team assumption (baseline): 1 senior full-stack engineer (lead) + 1 mid-level frontend engineer, working in parallel. Adjust ±20% for different team compositions.

Calendar weeks include: design, implementation, code review, unit + integration + concurrency tests, frontend E2E (Playwright), and engineer-side QA. **Excludes:** client review cycles, scope changes, and pilot/training (Phase 5).

Phase	Scope	Calendar weeks	Person-weeks	Billing share
Phase 1	Catalog (FR-001, FR-003) + Per-product pricing (FR-002) + User & role mgmt (FR-004) + Serialized receive (FR-005) + Sale (FR-007) + Stock-on-Hand dashboard for serialized (FR-012 partial) + Auth + offline PWA shell	4–5 wk	~7 pw	22%
Phase 2	QUANTITY receive with FIFO batches (FR-006) + Maintenance ticket (FR-008) + Project Pull workflow (FR-009) + Low-stock alerts (FR-016) + Monthly channel margin report (FR-013) + LINE/Viber notifications base (FR-018) + Append-only audit ledger (FR-019) + Stock-on-Hand QUANTITY drill-down (FR-012 complete)	7–9 wk	~12 pw	33%
Phase 3	Pricing override approval queue (FR-010) + Stock Adjustment (FR-011) + Holding-period report (FR-014) + Search & lookup (FR-015) + PDF + Excel exports (FR-017) + Override-exception report + Adjustments history report	3–4 wk	~5 pw	14%
Phase 4	Customer detail dashboard (FR-020 customer) + Project detail dashboard (FR-020 project) + Pull history view + Per-batch drill in Search + Role-tiered Pydantic schemas	3 wk	~4 pw	12%
Phase 5	Deployment (VPS provisioning, Traefik, Let's Encrypt, pg_dump cron, Sentry wiring) + LINE/Viber bot onboarding for first 10 users (D33) + BT scanner field tuning (D34) + Catalog seed import + 2-week parallel run with existing spreadsheet system (D38) + 1-day on-site training per office (BKK + YGN)	3–4 wk	~3 pw	14%
Phase 6	30-day post-launch bug-fix window (no charge); SLA: critical fixes within 1 business day, non-critical within 5 business days	4 wk calendar	~1 pw	5%
Total	All FR-001 → FR-020 + deployment + pilot + training + 30-day bug fix	24–28 wk (~5.5–6.5 months)	~32 pw	100%

Phase gates: Each phase ships usable software at its end. Client signs off on phase acceptance criteria (see [§13](#)) before the next phase begins. Sign-off triggers the phase invoice.

Change-order rate: New FRs or scope changes priced at KWG day rate, billed against a change-order log. PRD addendum required for any phase scope change.

11. Decision Log

Carry-over (D1–D31 from v2.5 — all locked)

See [PRD v2.5 §11](#) for full text. All 31 decisions stand unchanged.

New in v2.6 (D32–D38 — close all v2.5 open risks)

ID	Decision	Closes risk	Rationale
D32	Drop browser-side IDB encryption. Rely on device-level encryption + 12h auto-logout + 7-day queue cap.	R1 (browser-close data loss with session-keyed AES-GCM)	Cached data is operational inventory; no PII beyond what's already in the synced catalog. Session-keyed encryption introduced a data-loss failure mode (browser close → IDB unreadable) for negligible security gain at warehouse scale. Device-level encryption (iOS/Android, full-disk on laptops) is the modern baseline. <i>Alternatives considered:</i> (a) <i>passphrase-derived key — UX friction at every login;</i> (b) <i>document the failure mode and keep encryption — accepts known data loss;</i> (c) <i>drop encryption — chosen.</i>
D33	KWG performs first-time LINE/Viber bot enrollment for the initial 10 user accounts at deployment. Subsequent user enrollment is documented in the admin runbook.	R3 (per-user onboarding gate for LINE Messaging API + Viber Bot API)	Both APIs require users to first-message the bot before push works — this is platform behavior, not configurable. KWG handles the v1 cohort once; the client owns ongoing enrollment via a one-page runbook. <i>Alternatives considered:</i> <i>out-of-scope (client doesn't know to do it); KWG does all onboarding indefinitely (creates a permanent KWG ops dependency).</i> <i>One-time + runbook is</i>

			<i>the standard middle ground.</i>
D34	Reference scanner models: Honeywell Voyager 1602g + Zebra DS2208 (HID-mode). Other HID scanners best-effort; non-reference bring-up billable at day rate.	R4 (Bluetooth scanner model spec)	Both models are widely available in SEA, well-documented HID-mode behavior, and KWG has prior QA coverage. Pinning two reference models lets the quotation lock scanner-related QA scope. <i>Alternatives considered: client picks any HID scanner (variable QA cost); KWG supplies hardware (out of scope per §6.5 — client owns hardware procurement).</i>
D35	v1 tolerates duplicate customer records (e.g., from offline inline-creation racing during sync). Admin merge tool deferred to v1.1.	R5 (D34 in v2.5 — relabeled here to avoid confusion with v2.6's D34 scanner decision)	Yangon staff inline-creating customers offline can produce duplicates when two clients create the same customer before sync. Fuzzy-match-on-server adds complexity and false-positive risk in v1. Manual admin merge is acceptable at v1 volume (~5 new customers/month). <i>Alternatives considered: fuzzy match on sync (false positives); block inline create offline (defeats the offline goal). Defer to v1.1 is chosen.</i>
D36	part_batch.batch_no generation is race-safe via <code>pg_advisory_xact_lock(hashtext(date_yyyymmdd '-' sku))</code> within the receipt transaction. Sequence: <code>COALESCE(MAX(suffix), 0) + 1</code>.	R6 (concurrent same-SKU same-day receipts racing on <code>MAX(suffix)+1</code>)	Postgres advisory locks are session/transaction-scoped, deadlock-free under deterministic key hashing, and cost ~microseconds. Cleaner than a per-(date, sku) sequence (which would require dynamic sequence creation).

			<i>Alternatives considered:</i> <i>per-(date, sku)</i> <i>sequence (ops complexity);</i> <i>SERIALIZABLE isolation (perf hit on all writes);</i> <i>accept the race (occasional duplicate batch_no → constraint violation surfaces).</i>
D37	§10 phasing rewritten with realistic calendar weeks. Team baseline: 1 senior + 1 mid in parallel. Total 24–28 calendar weeks (~5.5–6.5 months) including deployment, pilot, training, and 30-day post-launch fix window.	Timeline ambiguity in v2.5 §10 (which said "~9 weeks" without team-size or scope-of-included-activities context)	v2.5's 9-week phasing did not specify person-weeks vs calendar weeks and excluded testing, deployment, training, and post-launch support. The revised phasing is the engagement contract for the KWG ↔ CastraNova SOW. Billing share per phase fixed; change orders required for scope changes. <i>Alternatives considered:</i> <i>keep v2.5 wording with a footnote (preserves ambiguity); hourly T&M with no phase cap (poor client experience).</i> <i>Fixed-price per phase + change-order log is chosen.</i>
D38	2-week parallel run with existing spreadsheet system before retirement; 1-day on-site training per office (BKK + YGN). Scoped into Phase 5.	Pilot/training gap in v2.5 (mentioned in §11 but unscoped)	Parallel run catches reconciliation gaps before spreadsheet retirement; on-site training accelerates adoption and reduces post-launch support tickets. Both are standard ERP go-live patterns at this scale. <i>Alternatives considered:</i> <i>parallel run only (training-by-doing fails for the low-tech YGN persona); training only</i>

			(no rollback path if issues surface).
--	--	--	---------------------------------------

Still open

None. All v2.6 requirements are now locked. **PRD v2.6 supersedes v2.5 in full.**

12. Revision Log

Version	Date	Author	Changes
0.1 – 2.5	2026-05-25 → 2026-05-27	K W G	See v2.5 revision log for full detail. v2.5 locked D1–D31.
2.6	2026-06-02	K W G	Closes all v2.5 open risks (R1–R6) and the §10 timeline ambiguity. Locks D32–D38: drop IDB encryption (D32), formalize LINE/Viber bot enrollment ops (D33), reference scanner models Honeywell 1602g + Zebra DS2208 (D34), defer customer-merge tool to v1.1 (D35), race-safe batch_no via pg_advisory_xact_lock (D36), rewrite §10 phasing with realistic 24–28 wk calendar and explicit team baseline (D37), formalize 2-wk parallel run + 1-day per-office training in Phase 5 (D38). Adds §6.5 hardware spec, §6.8 observability, §13 acceptance-criteria index, §14 implementation notes for AI/KWG engineering. Expands §9 out-of-scope with explicit v1.1 deferrals. PRD v2.6 is fully locked and is the implementation contract for the KWG ↔ CastraNova SOW. Companion system design spec (docs/superpowers/specs/2026-05-23-castranova-pos-system-design.md) will be refreshed against v2.6 in its next revision; existing spec D32–D38 (system-design-layer) will be renumbered S1–S7 to avoid clash.

13. Acceptance Criteria Summary

Per-FR acceptance map — used for phase sign-off gates.

FR	Acceptance test (one per FR — full criteria in §5)	Phase	Verified by
FR-001	Admin creates a SERIALIZED + a QUANTITY product; both appear in catalog; staff can scan SKU of QUANTITY product.	1	Manual + Playwright
FR-002	Receive 2 batches of one SKU at different costs; consume a quantity that spans both → 2 cost_line rows written with correct split.	2	pytest + DB inspection
FR-003	Admin creates customer + project + supplier; staff sees customer in dropdown but cannot edit. Staff inline-creates a	1	Playwright

	customer during a sale; row appears in admin's customer list.		
FR-004	Staff user cannot access /stock-adjustments POST (403); admin can. Staff user cannot see admin-only fields on /customers/{id}/dashboard.	1	pytest role guards
FR-005	Receive 3 serialized units; 3 unit rows + 3 unit_movement(RECEIVED) + 3 label PDFs produced.	1	pytest + Playwright
FR-006	Receive a QUANTITY batch; part_batch row created with auto batch_no, immutable received_qty, remaining_qty == received_qty. Concurrent same-SKU receipt produces sequential suffixes (D36).	2	pytest concurrency suite
FR-007	Sale of 5 QUANTITY units spanning batch boundary → sale + sale_line + part_movement + 2 cost_line written in one txn. Offline sale + reconnect → no duplicate (idempotency).	1+2	pytest + Playwright offline
FR-008	Open ticket → add 3 parts → close → 3 service_ticket_part + 1 part_movement per line + N cost_line written. Ticket closed_at populated.	2	pytest + Playwright
FR-009	Admin creates pull with 1 SERIALIZED + 1 QUANTITY line. Staff fulfills SERIALIZED, marks QUANTITY short → pull state = SHORT, admin notified, both actor_user_id recorded.	2	pytest + Playwright + notification mock
FR-010	Override at 3% (below 5% threshold) auto-approves. Override at 10% routes to admin queue; admin approves; sale completes. Override at 10% rejected; sale blocked.	3	pytest + Playwright
FR-011	Admin negative adjustment of 8 units spanning 2 batches → FIFO consumes oldest first, both batches recorded on adjustment. Positive adjustment of 5 units → new ADJ-### batch created.	3	pytest
FR-012	Dashboard loads stock-on-hand at P95 < 10s with 500 SKUs + 5000 batches in DB. Per-batch drill expands inline.	1+2	Performance test
FR-013	Run channel margin report for a month with mixed Sale/Maintenance/Project consumption → revenue + COGS + margin per channel correct vs hand calc.	2	pytest snapshot
FR-014	Holding-period report flags units > 90 days; per-batch ages visible; SKU rollup shows oldest batch age.	3	pytest
FR-015	Search serial → full lifecycle returned in chronological order. Search SKU → per-batch history + consumption history with batch attribution.	3	pytest
FR-016	Set min-stock = 10 on a SKU; consume to 9 → LINE + Viber alerts fire to opted-in admins. Bulk-edit screen sets thresholds on 20 SKUs at once.	2+3	pytest + notification mock + Playwright

FR-017	Each of the 10 list/report views exports correctly to PDF and Excel; column headers + row data match on-screen view.	3	pytest snapshot
FR-018	Per-user opt-out of LINE for low-stock event → user receives no LINE on next alert but still receives Viber. Bot enrollment runbook documented.	2	pytest + notification mock
FR-019	DB-level REVOKE UPDATE, DELETE blocks app-role from editing unit_movement, part_movement, cost_line, price_change. Audit log filter by user + date + event-type returns correct rows.	1+2	DB integration test
FR-020	Admin sees revenue/COGS/margin on customer detail. Staff sees no financial columns (absent, not blank) on the same endpoint — verified via raw HTTP response inspection. Export buttons disabled for staff.	4	pytest + Playwright + manual network inspection

14. Implementation Notes for AI / KWG Engineering

Build order

- Phase 1 backend:** Migrations M001–M008 + M010 + M015 → models in `app/models.py` → `crud.py` extensions → routes (`products` , `customers` , `suppliers` , `users` , `receipts/serialized` , `sales` , `auth`).
- Phase 1 frontend:** TanStack Router scaffolding for admin + staff role-based shells → catalog screens → receive-serialized → sale (online first) → serialized stock dashboard → offline shell (vite-plugin-pwa + TanStack Query persister).
- Phase 2 backend:** Migrations M009 + M011–M013 + M016–M019 → FIFO consumer in `crud.py` with `pg_advisory_xact_lock` + `SELECT ... FOR UPDATE ORDER BY received_at, id` → maintenance + project-pull routes → notification dispatcher (tenacity + LINE/Viber clients) → append-only ledger DB role.
- Phase 2 frontend:** Quantity receive flow → maintenance ticket → project pull queue + fulfillment + short marking → low-stock list + min-stock CRUD → monthly margin report (server-side aggregation).
- Phase 3:** Override approval workflow → Stock Adjustment screens + FIFO-aware delta handling → holding-period + search + override-exception reports → PDF (reportlab) + Excel (openpyxl) exporters.
- Phase 4:** Customer + project detail pages → role-tiered Pydantic `*Public` schemas (admin vs staff) → pull history → per-batch drill in search.
- Phase 5:** Hostinger VPS deploy (Traefik + Let's Encrypt + Sentry DSN wiring) → seed import → LINE/Viber bot onboarding for 10 users → BT scanner field tuning → 2-week parallel run + training.

File structure (existing template — see [CLAUDE.md](#))

```

backend/
  app/
    api/routes/      # one file per resource (products.py, sales.py,
project_pulls.py, ...)
    alembic/versions/ # one Alembic revision per migration (M001–M019 per spec §9)
    models.py        # SQLAlchemy + *Public/*Create/*Update schemas
    crud.py           # all DB access
    core/             # config, security, db engine

```

```

frontend/
  src/
    routes/          # TanStack file-based routes (admin/, staff/)
    components/      # shadcn/ui primitives + app components
    client/          # AUTO-GENERATED – never hand-edit
    lib/             # offline queue helpers, scanner adapter, html5-qrcode wrapper

```

Critical implementation details

- **All monetary fields are `NUMERIC(12,2)`** — never `FLOAT` (precision bugs in FIFO math).
- **FIFO concurrency:** `SELECT ... FOR UPDATE ORDER BY received_at, id` for batch locks; `pg_advisory_xact_lock` for `batch_no` sequencing. Test with `pytest + ThreadPoolExecutor`.
- **Idempotency keys are required** on every mutation request schema (sale, service ticket, project pull fulfill, stock adjustment, receive). DB-level `UNIQUE` constraint per ledger table.
- **Append-only ledgers:** `REVOKE UPDATE, DELETE ON unit_movement, part_movement, cost_line, price_change FROM app_role` — enforce at the DB, not just in code.
- **Role-tiered schemas:** Two `*Public` Pydantic variants per role-sensitive endpoint (FR-020); route dispatches by `current_user.role`. Never rely on frontend hiding.
- **Offline queue:** TanStack Query mutation queue persisted via `createAsyncStoragePersister(idb=keyval)` — **no app-level encryption** (D32). 7-day cap via mutation `submittedAt` filter; `STALE` flag for items past cap.
- **Conflict resolution:** First-write-wins on the server (`SELECT ... FOR UPDATE` on `unit` row before state-change movements). Loser receives 409 + current state JSON; frontend translates to "already sold by [X] at [time]".
- **Notifications:** LINE Messaging API push endpoint `https://api.line.me/v2/bot/message/push` with user IDs from `user.line_user_id` captured at bot first-message webhook. Viber Bot API push at `https://chatapi.viber.com/pa/send_message`. Tenacity retry: 3 attempts, exponential backoff (1s, 5s, 25s). Permanent failures → `notification_log`.

Libraries to use

- **Backend:** `SQLModel` (template), `Alembic` (template), `tenacity` (retry), `reportlab` (PDF), `openpyxl` (Excel), `python-barcode` (Code128 generation), `slowapi` (rate limit), `sentry-sdk`.
- **Frontend:** TanStack Router + Query (template), shadcn/ui (template), vite-plugin-pwa (PWA shell), `idb-keyval` (IDB), `@tanstack/query-async-storage-persister`, `html5-qrcode` (camera fallback scanner), `react-hook-form` + `zod` (forms), `@sentry/react`.

Libraries to avoid

- Heavy state libraries (Redux, Zustand) — TanStack Query covers server state; React local state covers UI state at this scale.
- Bcrypt directly — use `pwdlib` from the template.
- Custom JWT impl — `PyJWT` from the template.
- Service worker frameworks beyond vite-plugin-pwa — adds complexity without payoff at this scale.

Common pitfalls

- **Batch FIFO across transactions:** Without `FOR UPDATE`, two concurrent consumers can both read `remaining_qty = 10`, both write `-5`, and end at `0` after a 15-unit total draw. Always lock. Always re-read after lock.

- **batch_no race:** Don't compute `MAX(suffix) + 1` without the advisory lock; under concurrent receipt of same SKU on same date, both will compute the same suffix.
- **Offline duplicate detection:** Idempotency key must be generated client-side and persisted with the queued mutation, not regenerated on replay. Use `crypto.randomUUID()` at form submit.
- **HID scanner input handling:** Listen for the terminator character (CR or LF) on the input's `keydown`, not on `change` — `change` fires only on blur and misses fast scans.
- **PWA service worker caching:** Cache the app shell aggressively, but **never cache API responses** beyond what TanStack Query manages in memory + IDB. Cached stale API responses on offline screens lead to silent data drift.
- **TanStack Query mutation queue:** `resumePausedMutations()` must run after the auth token is rehydrated, not before — otherwise queued mutations fail with 401.
- **Append-only REVOKE** : Apply via Alembic migration M019, **after** all FK tables exist. Test in staging that the app role genuinely cannot DELETE from these tables.

Testing approach

- pytest baseline: 80% coverage on `crud.py` + routes (template default).
- Dedicated concurrency suite for FIFO + batch_no using `ThreadPoolExecutor` — 10 concurrent writers on the same SKU.
- Playwright covers 5 critical user journeys: receive-serialized, receive-quantity, sale (online), sale (offline → reconnect → replay), project-pull fulfill (with short line).
- Notification tests use a mock LINE/Viber client; verify retry behavior and opt-out respect.
- Skip: backend file-upload tests (no upload feature), email tests beyond template defaults.

v2.6 is the implementation contract for the KWG ↔ CastraNova engagement. Any divergence between code and this PRD is a bug. Any divergence between this PRD and v2.5 is captured in D32–D38 + the v2.6 revision log; no further open items.